

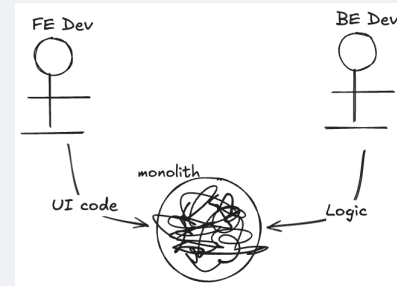
Light-weight Rust orchestrator for data processing tasks

Agenda

- Problem
- Constraints
- Solution
- Showcase
- Wrap Up
- QnA

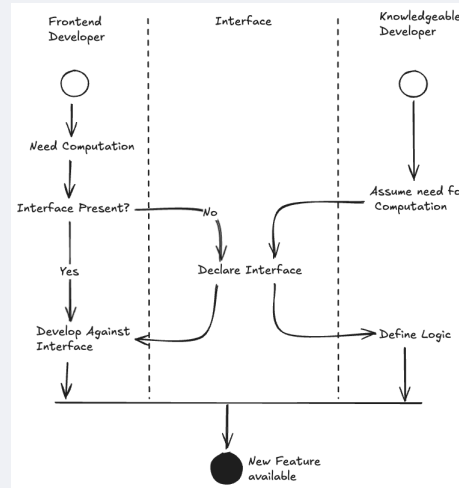
Sybila

- Complex algorithms, low-level libraries
- UI to expose the functionality (AEON)
- Business Logic without dedicated place
- Frontend Developers' domain understanding?



- Dev sync overhead
- Duplicate efforts
- Cognitive load
- Perf bound by user's HW
- Difficult to develop in parallel

Concept: Contract-first approach



Benefits

- Parallel development, client-server architecture
- Less sync between devs, reduced cognitive load

Constraints & Requirements

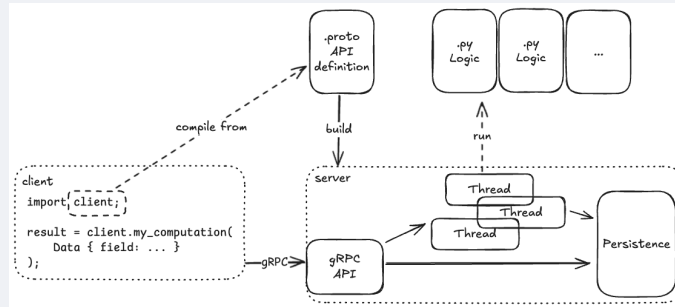
Need

- Long-running Computations (Hours)
- Large I/O (GBs)
- Logic in Python
- Integrate with Rust, TypeScript, Python
- Deployment local && remote

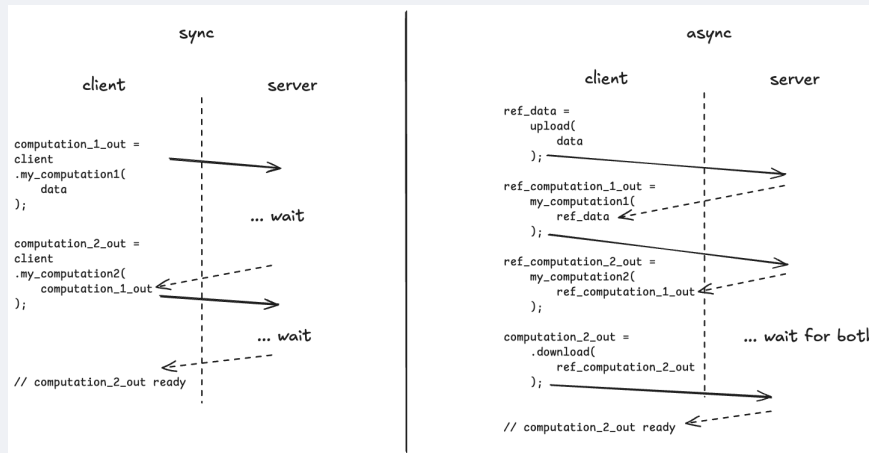
Want

- Type safety
- Composing Computations (conveniently)

High-level Design



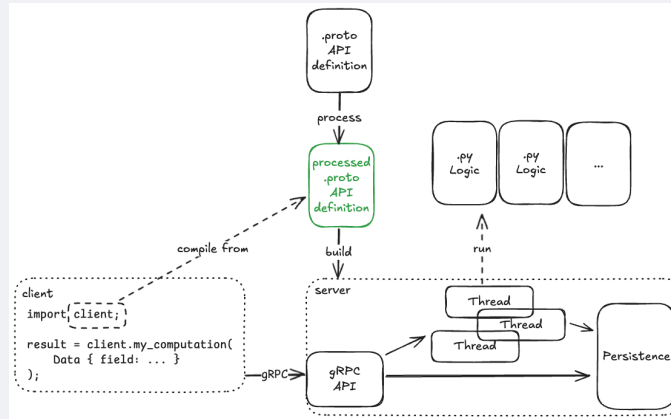
Addressing I/O size & Composition -- async API



Benefits

- Less traffic
- Reusable I/O data
- Orchestration on Server (Client may go offline)

High-level Design -- Vol 2



Showcase

API Specification

```
// bio-engine.proto

// A message holding very interesting input
message MyInput {
    string interesting_input_field = 1;
}
// A message holding very interesting output
message MyOutput {
    string interesting_output_field = 1;
}

service Engine {
    // A computation performing very interesting logic
    rpc MyComputation (MyInput) returns (MyOutput);
}
```

Info

- Build the project for artifacts to be generated
- Now, Frontend and Logic can be developed independently

API Specification -- Side Note

```
// bio-engine.proto

message RefFoo {} // <- problematic

// A message holding very interesting input
message MyInput {
    string interesting_input_field = 1;
}

// A message holding very interesting output
message MyOutput {
    string interesting_output_field = 1;
}

service Engine {
    // A computation performing interesting logic
    rpc MyComputation (MyInput) returns (MyOutput);
}
```

```

x [BE-004] Used a reserved prefix `Ref` in
`RefFoo`
4 | [.../proto/bio-engine.proto:5:9]
5 | message RefFoo {} // <- problematic
  |      └─ this message
6 |
help: Try renaming the message

```

Python Logic

```
# my_computation_handler.py
import api_processed_pb2
import sys

def logic_handler(
    request: api_processed_pb2.MyInput,
) -> api_processed_pb2.MyOutput:
    # compose the response and return it
    response = api_processed_pb2.MyOutput()
    response.interesting_output_field = \
        f"received:
{request.interesting_input_field}"
    return response

def main():
    binary_request = sys.stdin.buffer.read()
    # ...
```

```
def logic_handler(
    request: api_processed_pb2.MyInput,
) -> api_processed_pb2.MyOutput:
    # compose the response and return it
    response = api_processed_pb2.MyOutput()
    response.in
    return resp[0] interesting_output_field
    # INTERESTING_OUTPUT_FIELD_FIELD_NUMBER
```

Info

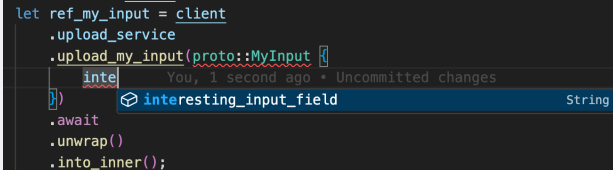
- Server can now be deployed locally or remotely

Rust Client

```
let ref_my_input = client
    .upload_service
    .upload_my_input(proto::MyInput {
        interesting_input_field: "content".into(),
    })
    .await
    .unwrap()
    .into_inner();

let ref_my_output = client
    .engine
    .my_computation(proto::ReqMyInput {
        req: Some(ref_my_input),
        ..Default::default()
    })
    .await
    .unwrap()
    .into_inner();

let my_output = client.download_service.
download_my_output(ref_my_output);
```



```
let ref_my_input = client
    .upload_service
    .upload_my_input(proto::MyInput {
        interesting_input_field: "content".into(),
    })
    .await
    .unwrap()
    .into_inner();
```

The screenshot shows an IDE with a tooltip for the `interesting_input_field` field. The tooltip text is "You, 1 second ago • Uncommitted changes". The field name `interesting_input_field` is highlighted in blue, and the type `String` is shown to the right.

Rust Client -- DSL

```
let my_output = crate::pipe!(  
    client,  
    proto::MyInput { interesting_input_field: "content".into() }  
=>  
    upload_service::upload_my_input  
    |> engine::my_computation  
    |> download_service::download_my_output  
) .await.unwrap();
```

Rust Client -- DSL Vol 2

```
let my_output = crate::pipe!(  
    client,  
    proto::MyInput { interesting_input_field: "content".into() }  
=>  
    [chunk_size: 2048] upload_service::upload_my_input_stream  
    |>[time_limit_seconds: 42] engine::my_computation  
    |> download_service::download_my_output  
) .await.unwrap();
```

Rust Client -- DSL error

```
let my_
cli
pro
diagnostic)
=>
[ch View Problem (⌘F8) No quick fixes available
|> [time_limit: 42] engine::my_computation You, 1 second ago
|> download_service::download_my_output
).await.unwrap();
```

Wrapping Up

Questions?

Thank you!